

毕设论文 — 输入模板 v7

workflows: 分 3 轮使用，每轮指定 Part 编号 + 填写该 Part 核心信息 → Claude 输出对应章节（md 格式，正文内容）。

目标字数: 约 1.6 万字

核心原则: 你只填项目核心思路/方案/素材相关的内容，其余（论文常规话术、过渡段落、通用描述）Claude 自由发挥。

图片处理规则: 你输入的**每一张图片** Claude 都会插入论文相应章节，并对图片内容展开描述。

图片命名规则: 图片放在 md 文件同级目录下，按以下命名约定，Claude 在论文中按顺序自动引用：

- 任务书：任务书.png
- 系统框架图：系统框图.png
- 模块图（用于第二章器件选型）：模块_1.png、模块_2.png
- 原理图（用于第三章硬件设计）：原理图_1.png、原理图_2.png
- 软件章节：软件流程图_1.png、软件流程图_2.png
- 机械章节：机械_1.png、机械_2.png
- 测试章节：测试_1.png、测试_2.png

全局信息（Part 1 一次性填写，三轮复用）

G1. 项目背景与任务需求书

粘贴项目背景描述和任务需求书全文，或贴任务书图片。

Claude 会从中提取信息用于摘要、绪论、设计目标、创新点、关键词等部分。

项目名称:

任务书:

1. 本课题的研究内容、重点及难点 本研究基于 STM32 单片机平台，融合二维码识别、传感器检测与电机驱动技术，构建集包裹信息采集、智能分拣控制、机械执行与人机交互于一体的智能分拣系统，实现多目的地、多货物种类的精准分拣，满足漏检或错检率$\leq 5\%$的设计要求。 (1) 集成二维码扫描模块与传感器，完成包裹编号、目的地、重量等信息的采集、解析与格式转换，搭建稳定的基础数据获取链路，为分拣决策提供可靠数据支撑。 (2) STM32 核心控制体系搭建，包括 STM32 与各外设的硬件适配、驱动开发，以及分拣控制算法设计，提取图片文字，确保包裹按预设规则自动输送至对应 (3) 交互界面与上位机系统开发，梳理系统总体架构与软件流程，搭建样机并开展调试验证，保障系统各项功能达标。 数据及要求： (1)能够分拣的目的地≥ 3处； (2)分拣货物的种类≥ 2种； (3)漏检或错检率$\leq 5\%$； 研究重点： 以STM32为核心，通过高精度识别模块、实时数据处理算法与智能控制机制的研发，保障智能物流分拣系统的分拣准确性、数据时效性与输送精准性。 (1) 开发基于STM32的高精度包裹识别与分类模块，融合二维码识别与传感器检测技术，确保分拣过程的准确性。 (2) 基于STM32设计分拣数据的实时采集与动态处理程序，实现物流分拣数据的快速获取、分析与存储。 (3) 依托STM32设计智能分拣控制机制，编写控制算法使包裹根据目的地信息自动输送至对应分拣位置。 研究难点： 围绕STM32控制的智能物流分拣系统，需攻克分拣机构机械设计的适配性难题与软件控制算法的精准性、稳定性设计挑战。 (1) 基于STM32控制需求，完成包裹分拣机构的机械结构设计，需兼顾分拣动作的灵活性与硬件驱动的适配性。 (2) 针对STM32的运算特性，设计包裹分拣机构的软件控制算法，需解决算法的实时性、精准性与系统兼容性问题。
--

【或在此粘贴文字版任务需求书】

G2. 论文目录

默认目录如下，可按项目实际修改。Claude 严格按最终目录输出。

#	摘要
#	Abstract
#	第一章 绪论
##	1.1 课题的研究背景与意义
###	1.1.1 (项目名称)国内外研究现状
###	1.1.2 (项目名称)的价值与意义
#	第二章 系统方案设计
##	2.1 需求分析
##	2.2 系统设计方案对比
##	2.3 系统总体设计方案（含器件选型）

```
# 第三章 硬件设计
## 3.1 原理图1设计（根据输入的原理图图片和备注自拟小节名称）
## 3.2 原理图2设计（根据输入的原理图图片和备注自拟小节名称）
## 3.x 原理图x设计（根据输入的原理图图片和备注自拟小节名称）...
# 第四章 软件设计
## 4.1 下位机软件设计
### 4.1.1 FreeRTOS 操作系统
### 4.1.2 STM32 基础外设配置
### 4.1.3 功能 1 软件设计（替换为任务书中具体的软件功能）
### 4.1.4 功能 2 软件设计（替换为任务书中具体的软件功能）
### 4.1.x 功能 x 软件设计（替换为任务书中具体的软件功能）...
## 4.2 上位机软件设计（若有）
### 4.2.1 上位机与下位机通信方案
### 4.2.2 前端用户 UI 设计
### 4.2.3 后端操作逻辑设计
# 第五章 机械外观设计（若有）
## 5.1 零件 1 设计（根据输入的零件名称和零件图片自拟小节名称）
## 5.2 零件 2 设计
## 5.x 零件 x 设计...
# 第六章 系统测试
## 6.1 功能 1 测试（根据输入的测试图片和图片描述自拟小节名称）
## 6.2 功能 2 测试
## 6.x 功能 x 测试...
# 第七章 总结与展望
```

G3. 输出格式要求

- 输出语言：中文（摘要附英文版）
- 文件格式：Markdown
- 章节编号：按 G2 目录输出，可适当对目录名称做出修改
- 图表编号：图 X-Y / 表 X-Y（章号-序号）
- 公式编号：(X-Y)
- 图片占位：用 `![图 X-Y 说明](.\图片名.png)` 引用，按本模板顶部命名规则
- 流程图/框图：用 Mermaid 代码块，可后续截图替换
- 字数：每 Part 按规划输出，允许 $\pm 10\%$ 浮动

G4. 额外备注（可选）

默认要求，可修改。这些规则在三轮输出中均严格执行。

1. 输出正文一定要口语化，尽可能多使用语气词或第一人称动词。比如：啊，吧，我，笔者，本人。
2. 文本中只使用逗号，句号，省略号，顿号。不要使用其它任何标点符号。
3. 需要有错别字，每 1000 字至少出现 2 处错别字。
4. 需要有语句表达不通顺的句子，每 5000 字至少有 2 处表达不通顺的语句。

【可在下方追加其他要求】

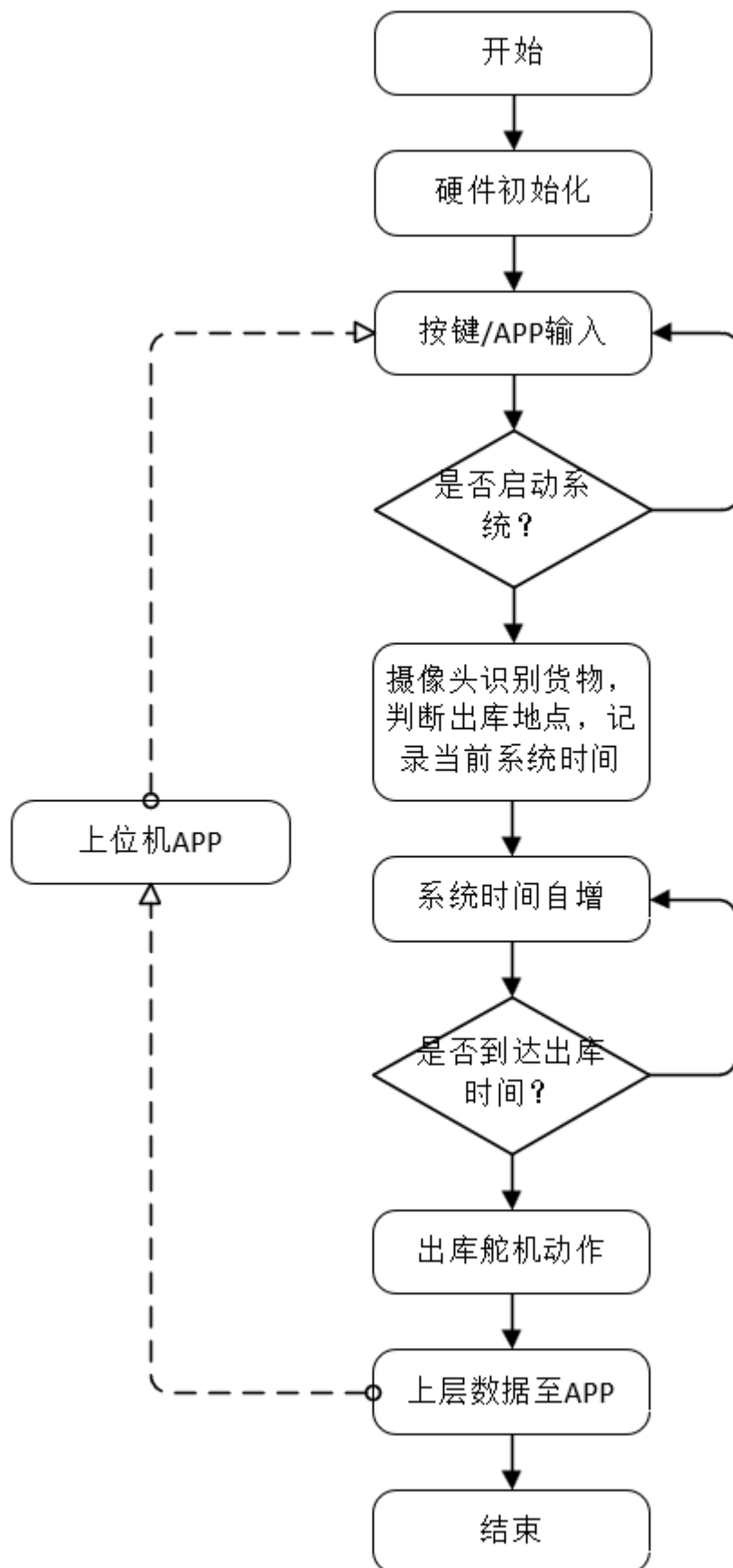
G5. 系统框图

Part 1（系统总体设计方案）和 Part 2（软件架构说明）均会引用此图。
Claude 据此图描述整机架构、模块连接关系、电源走向。

电源方案为本项目的固定方案：7.4V 锂电池供电 → 两路 LDO 稳压输出 5V + 一路 DCDC 升压 12V（若有）。

Claude 会根据系统框架图自动判断本项目用到了哪些电源支路。

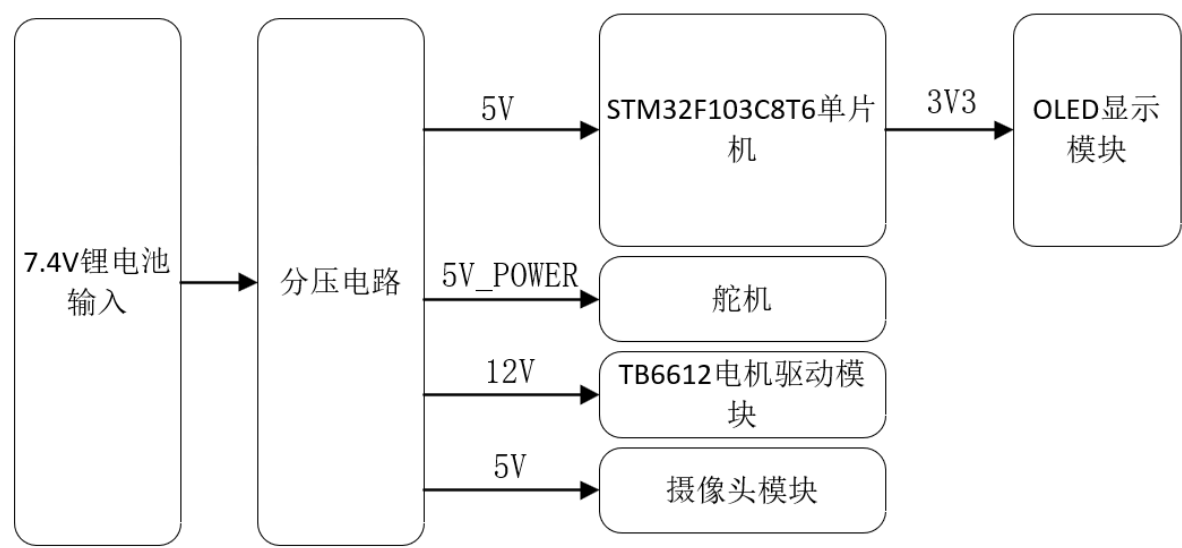
系统总体流程图：



描述（可选）：

系统方案是：摄像头模块识别物块上的二维码，获取该物块的出站信息（哪个舵机动作），并记录扫码时间，计算出出站时间（舵机位置固定，传送带速度固定，则物块到舵机处的时间也就固定）。物块随着传送带运动，到达出站时间后相应舵机动作，把物块推出传送带，即完成一次分拣。共三个位置，两个舵机，每个舵机代表一个出站位置，两个舵机都不动作即为第三个位置。

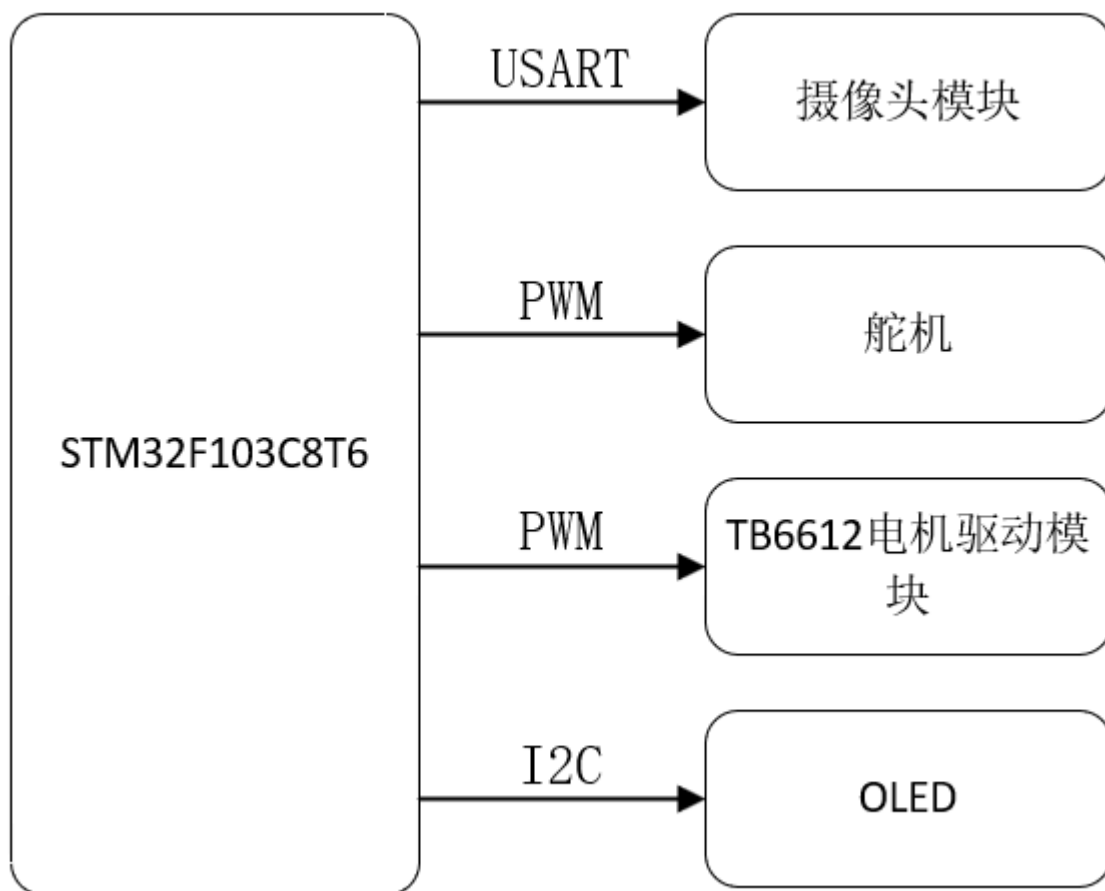
系统电源树：



描述（可选）：

分压电路输出两路5V LDO，一路12Vdcdc。可以说明舵机和其它的5V分开供电是因为舵机是感性负载。12V升压是为了传送带电机供电恒定，不受电池影响，从而速度恒定

系统结构图：



Part 1：摘要 + 绪论 + 系统方案设计（含器件选型）

触发指令：输出 Part 1

输出章节：摘要 / Abstract / 第一章绪论 / 第二章系统方案设计

预计字数：4000~5000 字

依赖：G1~G5 + 本 Part 输入

P1.1 国内外研究现状素材（可选但强烈建议）

列举 2~4 个相关方案/产品/论文，Claude 据此撰写国内外现状段落。

不填则 Claude 按行业常识生成通用描述，真实性较弱。

格式随意，例：

- 国外：某产品/论文，简述特点和不足
- 国内：某产品/论文，简述特点和不足

【在此填写】

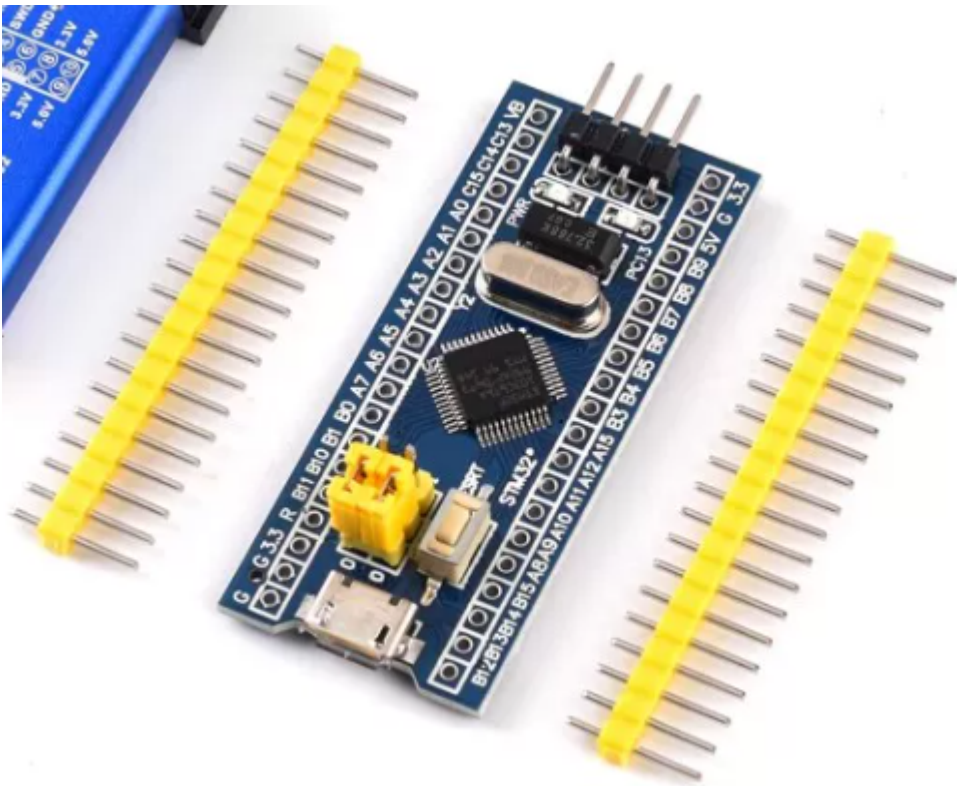
P1.2 方案对比（可选）

如果前期做过方案对比，简述被否方案和最终方案。
不填则 Claude 按"通用方案 vs 本设计"格式编 1~2 个对比方案。
【在此填写，留空则 Claude 自由发挥】

P1.3 器件选型素材（用于 2.3 系统总体设计方案）

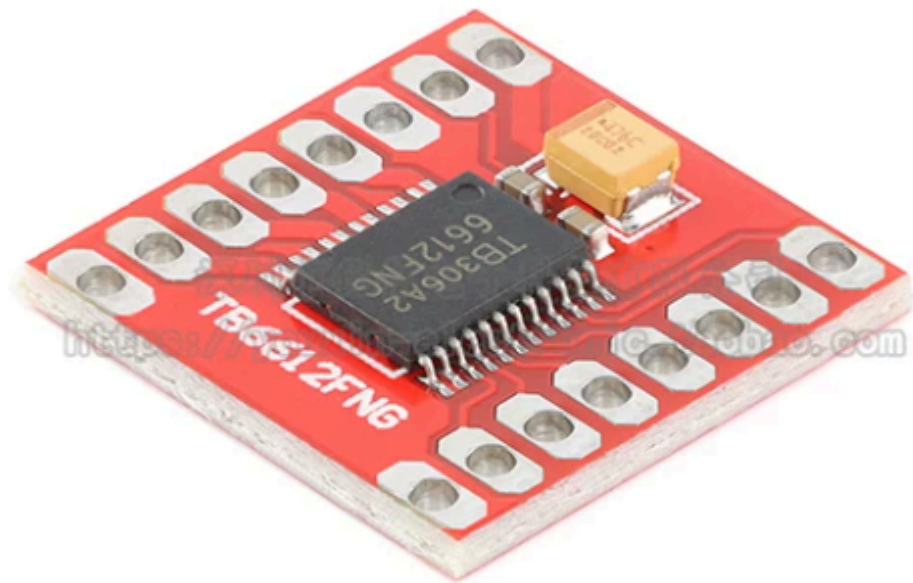
提供本项目所用模块的图片 + 简短描述。Claude 在 2.3 节对每个模块做介绍，包括：型号、关键参数、工作原理、在系统中的作用。
图片按 模块_1.png、模块_2.png 顺序命名。

模块 1：



- 描述：stm32最小系统

模块 2：



- 描述: tb6612电机驱动模块

模块 3:



- 描述: 0.96寸 白色 1315驱动 不焊接

模块 4:



- 描述: Sipeed MaixCAM RISC-V开发板 Linux语音 AI摄像头 视觉模块 电赛

模块 5:

[舵机实验常用款推荐]

SG90 9G经典舵机 QC模块遥控飞机马达



180°/360°

扭动力稳定

反应快

- 描述：SG90 9克舵机180度（小6孔配件）【常用推荐】

(按需添加更多)

Part 2：硬件设计 + 软件设计 + 机械外观设计

触发指令：输出 Part 2

输出章节：第三章硬件设计 / 第四章软件设计 / 第五章机械外观设计（若有）

预计字数：6000~7000 字

依赖：G1~G5 + 本 Part 输入

图片处理规则：本 Part 输入的每张图片，Claude 必须插入论文相应章节并展开描述（不少于 100 字/张）。

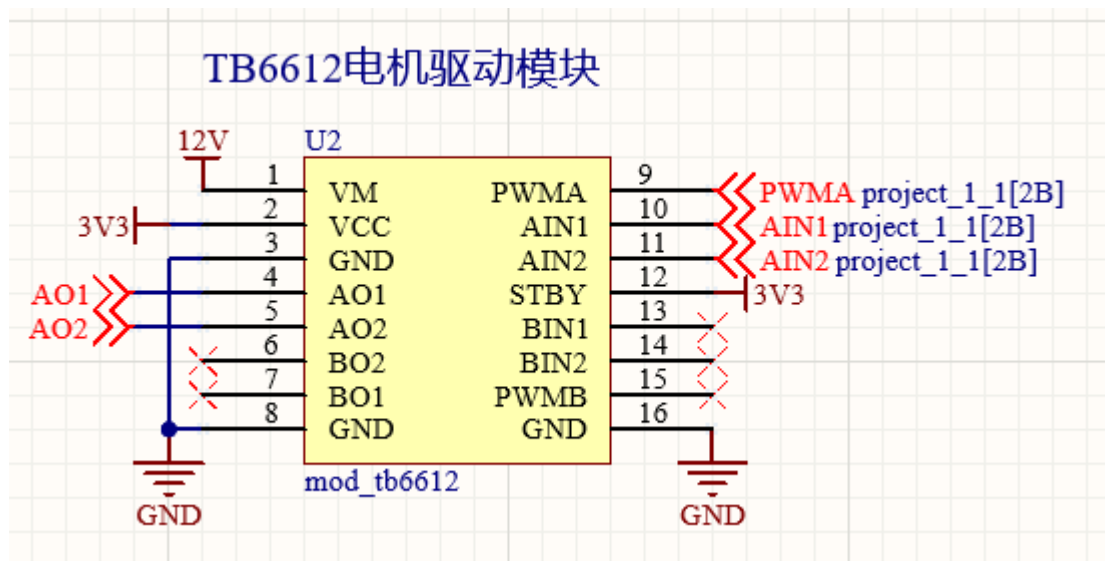
P2.1 原理图素材（用于第三章）

第三章按原理图数量拆分小节，每张原理图对应一个小节，小节名由 Claude 根据原理图内容和你的描述自拟。

Claude 会针对每张原理图描述其电路构成、关键器件、信号走向、设计要点。

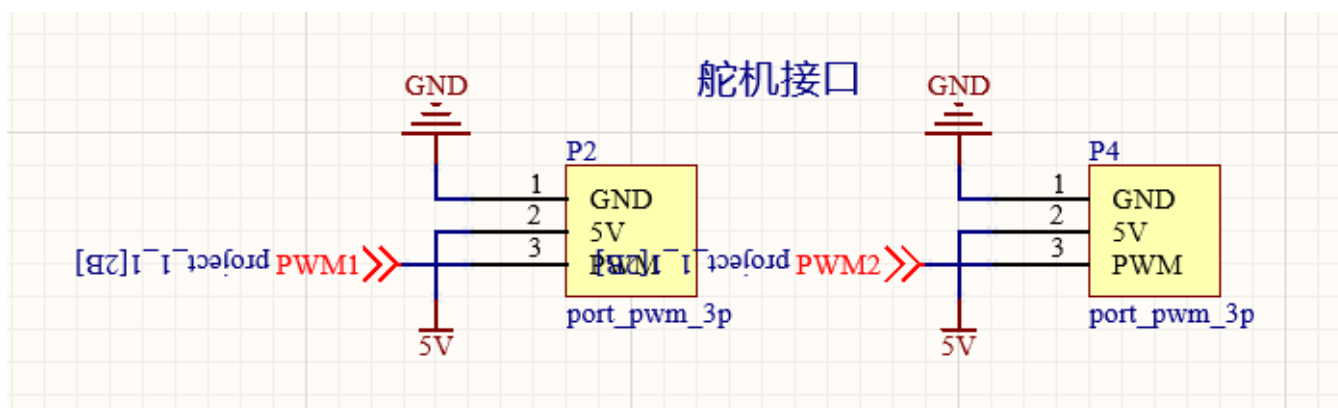
图片按 原理图_1.png、原理图_2.png 顺序命名。

原理图 1：



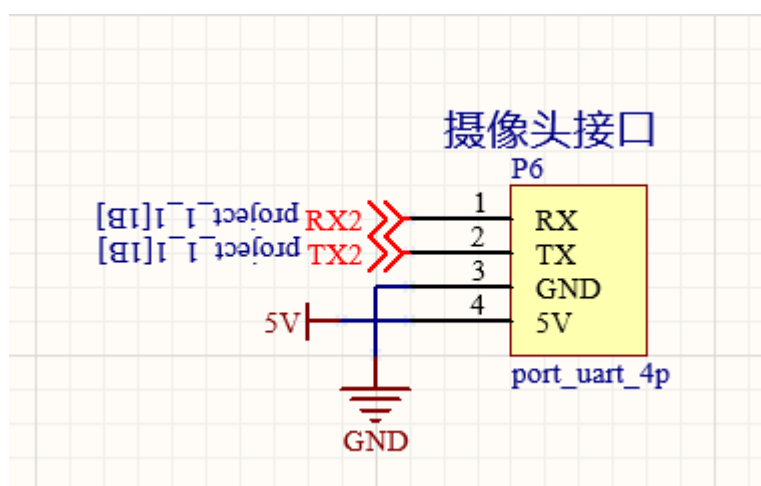
- 描述:

原理图 2:



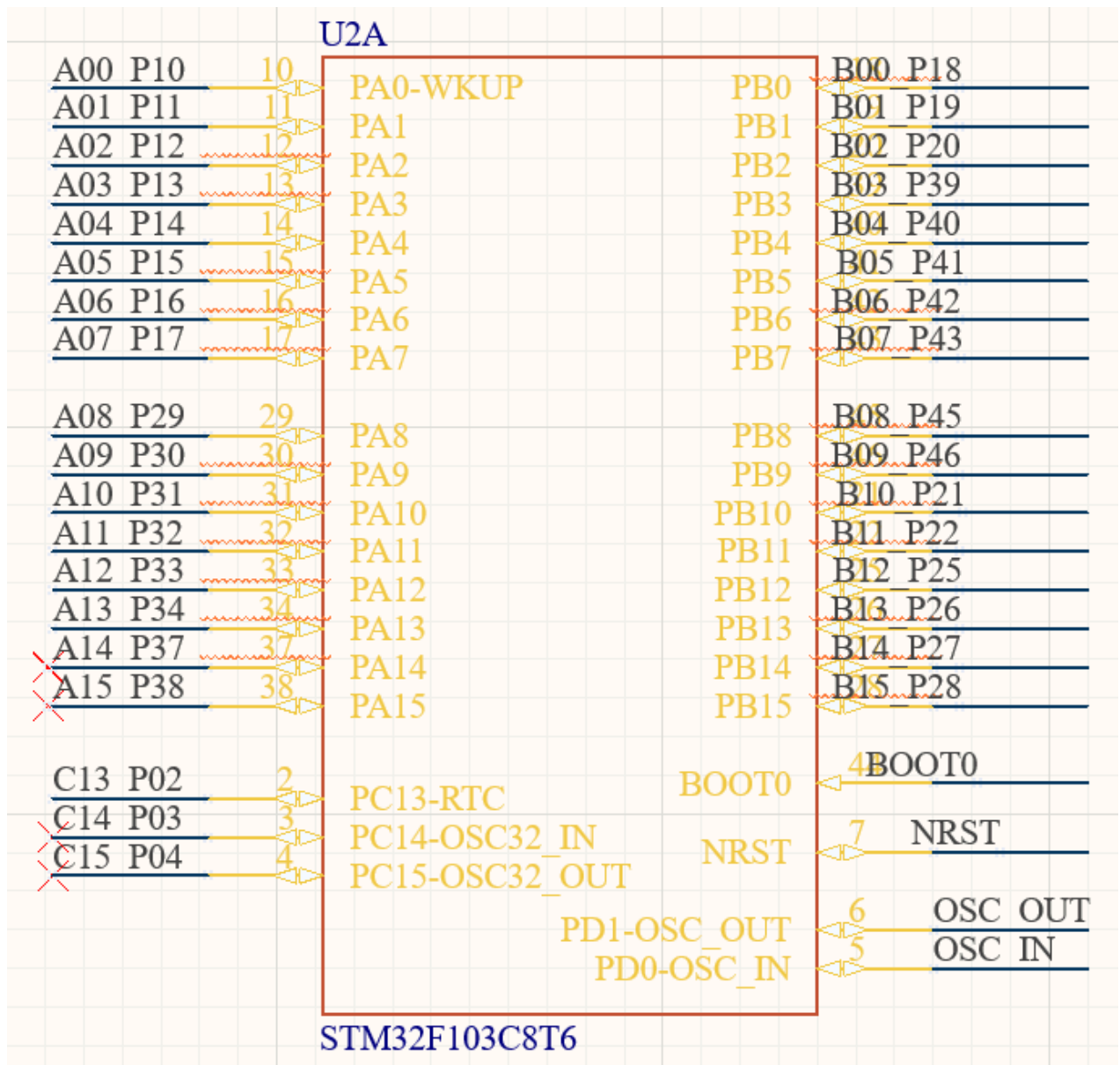
- 描述:

原理图 3:



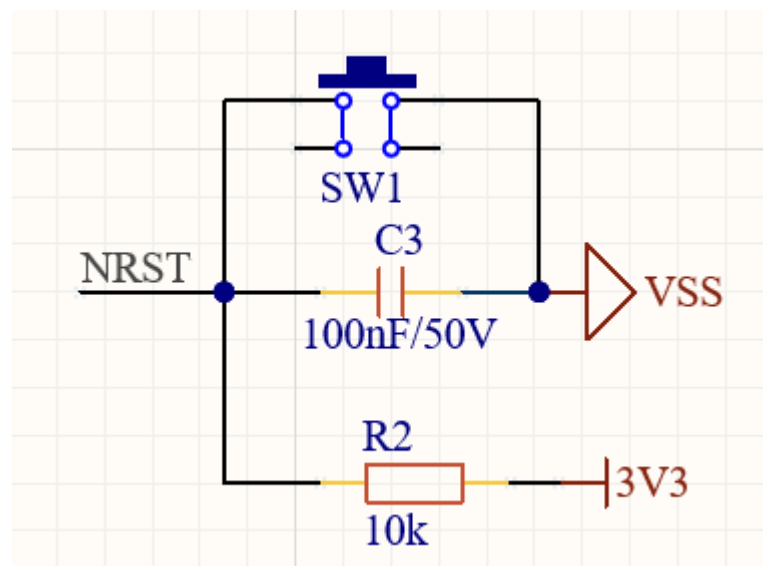
- 描述:

- 原理图 4:



- 描述：STM32最小系统

原理图 5：



- 描述：复位电路

(按需添加更多)

P2.2 软件素材（用于第四章）

每个功能小节只需提供"应用层代码 + 流程图 + 简短描述", Claude 据此撰写函数解释和流程描述。

P2.2.1 应用层代码

粘贴 task_user1.c / task_user1.h 全部代码, 或挑核心应用层代码。

task_system.c:

```
#include "task_system.h"
#include "driver_board.h"

systemTaskInfo_t systemTaskInfo;

uint8_t key1;
uint8_t remoteControl;

void systemTaskInit(){
    STDLIB_COMMON_PeriphInit();
    STDLIB_TIM_PwmInit();
    STDLIB_USART_Init();
    STDLIB_I2C_Init();
}

void systemTaskUpdata(void *argument){
    systemTaskInit();

    for(;;){
        systemTaskInfo.systemTaskCnt++;
        STDLIB_USART_Updata();

        if(DRIVER_BOARD_KeyIsPressed(BOARD_KEY1) || remoteControl){
            key1 = 1;
        }
        // if(!remoteControl){
        //     key1 = 0;
        // }

        if(systemTaskInfo.systemTaskCnt % 100 == 0){

        }

        osDelay(100);
    }
}
```

task_user1.c:

```
#include "task_user1.h"
#include "driver_oled.h"
#include "driver_tb6612.h"
#include "driver_board.h"
#include "driver_verison.h"
#include "task_system.h"
#include "func_appcom.h"

/*=====
 * 私有宏定义
 *=====*/

uint32_t timeA = 1400;
uint32_t timeB = 6200;

extern uint8_t key1;

/*=====
 * 私有变量
 *=====*/

/* 包裹列表 */
packInfo_t packList[PACK_MAX_NUM];
packInfo_t remoteInfo;

/* 系统状态 */
sysCtrl_t sysCtrl;

/* 电机当前速度，用于软启动斜坡控制 */
int16_t motorSpeed;

user1TaskInfo_t user1TaskInfo;

uint32_t QR;

/*=====
 * 私有函数
 *=====*/

/* 检测K210新识别结果，将包裹加入列表并计算触发时刻
 * 移植自uartRevPack(), 以k210RxFlag替代QRcodePack.revFlag
 * 每收到一帧驱动层置k210RxFlag=1, 此处读取后立即清零, 连续相同目的地均可记录 */
static void packRecognize(void){
    uint8_t *pHasNew = DRIVER_VERISON_HasNewData();
```



```

    if (pHasNew == NULL || *pHasNew == 0U) {
        return;
    }
    *pHasNew = 0U;

    packList[sysCtrl.packNum].aim      = versionRealData.field.var1;;
    packList[sysCtrl.packNum].isOutFlag = 0U;

    switch(versionRealData.field.var1){
        case 0x31: /* 目的地A: 延迟8s后触发舵机1 */
            packList[sysCtrl.packNum].outportTime = sysCtrl.taskCnt + CONVEYOR_DELAY_A;
            packList[sysCtrl.packNum].servoId     = STEER_SERVO_1;
            break;
        case 0x32: /* 目的地B: 延迟15s后触发舵机2 */
            packList[sysCtrl.packNum].outportTime = sysCtrl.taskCnt + CONVEYOR_DELAY_B;
            packList[sysCtrl.packNum].servoId     = STEER_SERVO_2;
            break;
        default: /* 目的地C: 不触发舵机, 直接通过 */
            packList[sysCtrl.packNum].outportTime = 0U;
            packList[sysCtrl.packNum].servoId     = SERVO_ID_NONE;
            break;
    }

    remoteInfo.aim = packList[sysCtrl.packNum].aim;
    remoteInfo.packNum = maixCamInfo.rxTotalCnt;
    remoteInfo.outportTime = packList[sysCtrl.packNum].outportTime;
    remoteInfo.servoId = packList[sysCtrl.packNum].servoId;

    sysCtrl.packNum++;
    if(sysCtrl.packNum >= PACK_MAX_NUM){
        sysCtrl.packNum = 0U;
    }
}

/* 遍历包裹列表, 到达触发时刻时使能对应舵机
 * 移植自sysControl(), 用>=比较避免舵机阻塞期间漏触发 */
static void sysControl(void){
    uint8_t i;
    for(i = 0U; i < sysCtrl.packNum; i++){
        if(packList[i].isOutFlag == 0U &&
            packList[i].servoId != SERVO_ID_NONE &&
            (int32_t)(sysCtrl.taskCnt - packList[i].outportTime) >= 0){

            if(packList[i].servoId == STEER_SERVO_1){
                sysCtrl.servoEnable[STEER_SERVO_1] = 1U;
            } else if(packList[i].servoId == STEER_SERVO_2){
                sysCtrl.servoEnable[STEER_SERVO_2] = 1U;
            }
            packList[i].isOutFlag = 1U;
        }
    }
}

```

```

}

/* 传送带电机控制：KEY1按下时斜坡加速，松开时立即停止
 * 移植自motorTaskUpdate()电机部分，使用TB6612_MOTOR_A驱动传送带 */
static void motorControl(void){
    if(1){
        if(motorSpeed < TB6612_SPEED_MAX){
            motorSpeed += MOTOR_SPEED_RAMP_STEP;
        } else {
            motorSpeed = (int16_t)TB6612_SPEED_MAX;
        }
        DRIVER_TB6612_MotorSetSpeed(TB6612_MOTOR_A, -motorSpeed);
    } else {
        motorSpeed = 0;
        DRIVER_TB6612_MotorSetSpeed(TB6612_MOTOR_A, 0);
    }
}

/* 舵机控制：仅在KEY1按下（电机运行）时触发，旋转500ms后自动停止
 * 移植自motorTaskUpdate()舵机部分，以Rotate360替代定时器计数 */
static void servoControl(void){
    uint8_t i;
    if(!key1){
        return;
    }
    for(i = 0U; i < STEER_SERVO_COUNT; i++){
        if(sysCtrl.servoEnable[i] == 1U){
            if(i == STEER_SERVO_1){
                DRIVER_STEER_Rotate360(STEER_SERVO_1, STEER_DIR_CW, 400); /* 顺时针旋转500ms
（拨开包裹） */
                DRIVER_STEER_Rotate360(STEER_SERVO_1, STEER_DIR_CCW, 390); /* 逆时针旋转500ms
（回到原位） */
            } else {
                DRIVER_STEER_Rotate360(STEER_SERVO_2, STEER_DIR_CCW, 400); /* 顺时针旋转
500ms（拨开包裹） */
                DRIVER_STEER_Rotate360(STEER_SERVO_2, STEER_DIR_CW, 380); /* 逆时针旋转500ms
（回到原位） */
            }
            sysCtrl.servoEnable[i] = 0U;
        }
    }
}

void oledControl(){
    DRIVER_OLED_Refresh();
    DRIVER_OLED_ShowString(5,5," PACKAGE SORT SYSTEM ");
    DRIVER_OLED_ShowString(5,15,"system time:");
    DRIVER_OLED_ShowNum(80, 15, systemTaskInfo.systemTaskCnt/10, 3);
    DRIVER_OLED_DrawPoint(100,20,1); //小数点
    DRIVER_OLED_ShowNum(102, 15, systemTaskInfo.systemTaskCnt%10, 1);
    DRIVER_OLED_ShowString(110,15,"s");

    DRIVER_OLED_ShowString(5,25,"----Packge Info----");

```

```

// DRIVER_OLED_ShowNum(100, 35, QRcodePack.id, 1, 24, 1);           //包裹编号

DRIVER_OLED_ShowString(5,35,"NUM:");
DRIVER_OLED_ShowNum(35,35, maixCamInfo.rxTotalCnt, 1);

DRIVER_OLED_ShowString(5,45,"system enable:");
DRIVER_OLED_ShowNum(100 ,45, key1, 1);
// switch(packList.source){
//     case 0x01: OLED_ShowString(50,35,"E",8,1); break;
//     case 0x02: OLED_ShowString(50,35,"F",8,1); break;
//     case 0x03: OLED_ShowString(50,35,"G",8,1); break;
// }
// OLED_ShowString(5,45,"aim:",8,1);
// switch(packList.aim){
//     case 0x01: OLED_ShowString(30,45,"A",8,1); break;
//     case 0x02: OLED_ShowString(30,45,"B",8,1); break;
//     case 0x03: OLED_ShowString(30,45,"C",8,1); break;
// }

}

/*=====
* 任务入口
*=====*/

void user1TaskInit(){
    DRIVER_OLED_Init();
    DRIVER_BOARD_KeyInit();
    DRIVER_VERISON_Init();
    /* 初始化360度舵机为停止状态 */
    DRIVER_STEER_Rotate360(STEER_SERVO_1, STEER_DIR_CW, 0U);
    DRIVER_STEER_Rotate360(STEER_SERVO_2, STEER_DIR_CW, 0U);
    FUNC_APPCOM_Init();
}

void user1TaskUpdata(void *argument){
    user1TaskInit();
    for(;;){
        user1TaskInfo.user1TaskCnt++;
        sysCtrl.taskCnt++;

        DRIVER_BOARD_KeyInfoUpdate();
        sysControl();
        motorControl();
        servoControl();
        DRIVER_VERISON_GetMaixCamInfo();
        packRecognize();

        /* OLED每100ms刷新一次，避免高频I2C传输拖慢控制循环 */
        if(sysCtrl.taskCnt % 50U == 0U){
            oledControl();
        }
    }
}

```

```

        if(sysCtrl.taskCnt % 5u == 0u){
            FUNC_APPCOM_TxUpdate();
            FUNC_APPCOM_RxUpdate();
        }

        osDelay(2);
    }
}

```

maixCam摄像头代码:

```

from maix import camera, display, image, app, uart, time

device = "/dev/ttyS0"
serial0 = uart.UART(device, 115200)

COOLDOWN_MS = 2000
DATA_MAX_LEN = 10

# -----
# 功能开关
# -----
ENABLE_CRC = True # CRC8校验开关: True=开启, False=关闭 (CRC字段发0x00)

cam = camera.Camera(320, 240)
disp = display.Display()

# -----
# CRC8 校验计算 (多项式 0x07)
# -----
def crc8(data):
    crc = 0x00
    for byte in data:
        crc ^= byte
        for _ in range(8):
            if crc & 0x80:
                crc = (crc << 1) ^ 0x07
            else:
                crc <<= 1
        crc &= 0xFF
    return crc

# -----
# 数据包格式 (固定16字节):
# [0]      0x55      包头1
# [1]      0xAA      包头2
# [2]      0x01      控制字段
# [3~12]   data      二维码内容 (10字节, 不足补0x00)
# [13]     CRC8      ENABLE_CRC=True时为校验值, False时为0x00

```

```

# [14]    CNT    包计数 (0x00~0xFF 循环递增)
# [15]    0xFF    包尾
# -----
pkt_cnt = 0

def send_packet(payload_str):
    global pkt_cnt

    data    = payload_str.encode('utf-8')[:DATA_MAX_LEN]
    data    = data + bytes(DATA_MAX_LEN - len(data))
    pre     = bytes([0x55, 0xAA, 0x01]) + data    # [0~12] 共13字节

    # ★ CRC字段: 根据开关决定计算还是填0x00
    crc     = crc8(pre) if ENABLE_CRC else 0x00

    cnt     = pkt_cnt & 0xFF
    pkt_cnt += 1

    pkt     = pre + bytes([crc, cnt, 0xFF])

    serial0.write(pkt)

    hex_str = " ".join("{:02X}".format(b) for b in pkt)
    print("=" * 40)
    print("已发送数据包")
    print("HEX: " + hex_str)
    print("QR:  " + payload_str)
    print("CRC: {:02X} ({}). CNT: {:d}".format(
        crc,
        "校验值" if ENABLE_CRC else "已禁用",
        cnt
    ))
    print("=" * 40)

# -----
# 状态机
# -----
STATE_IDLE    = 0
STATE_SENT    = 1
STATE_COOLDOWN = 2

state          = STATE_IDLE
cooldown_start = 0
last_payload   = ""

# -----
# 主循环
# -----
while not app.need_exit():
    img = cam.read()

    detected_payload = None
    qrcodes = img.find_qrcodes()

```

```

for q in qrcores:
    detected_payload = q.payload()
    corners = q.corners()
    for i in range(4):
        img.draw_line(
            corners[i][0], corners[i][1],
            corners[(i+1)%4][0], corners[(i+1)%4][1],
            image.COLOR_RED
        )
    break

now = time.ticks_ms()

if state == STATE_IDLE:
    if detected_payload is not None:
        send_packet(detected_payload)
        last_payload = detected_payload
        state = STATE_SENT
        cooldown_start = now

elif state == STATE_SENT:
    if detected_payload is None:
        state = STATE_COOLDOWN
        cooldown_start = now

elif state == STATE_COOLDOWN:
    elapsed = now - cooldown_start
    if elapsed >= COOLDOWN_MS:
        state = STATE_IDLE
        last_payload = ""
        print("COOLDOWN结束 -> IDLE")

if detected_payload is not None:
    img.draw_string(0, 0, "Scan: " + detected_payload[:10], image.COLOR_BLUE)
else:
    img.draw_string(0, 0, "Scan: None", image.COLOR_WHITE)

state_str = ["IDLE", "SENT-WAIT", "COOLDOWN"][state]
state_color = [image.COLOR_WHITE, image.COLOR_GREEN, image.COLOR_YELLOW][state]
img.draw_string(0, 20, "State: " + state_str, state_color)

if state in (STATE_SENT, STATE_COOLDOWN) and last_payload:
    img.draw_string(0, 40, "Sent: " + last_payload[:10], image.COLOR_GREEN)
else:
    img.draw_string(0, 40, "Sent: --", image.COLOR_WHITE)

if state == STATE_COOLDOWN:
    elapsed = now - cooldown_start
    remaining = max(0, COOLDOWN_MS - elapsed)
    img.draw_string(0, 60, "CD: {:.1f}s".format(remaining / 1000.0), image.COLOR_YELLOW)
else:
    img.draw_string(0, 60, "CD: --", image.COLOR_WHITE)

```

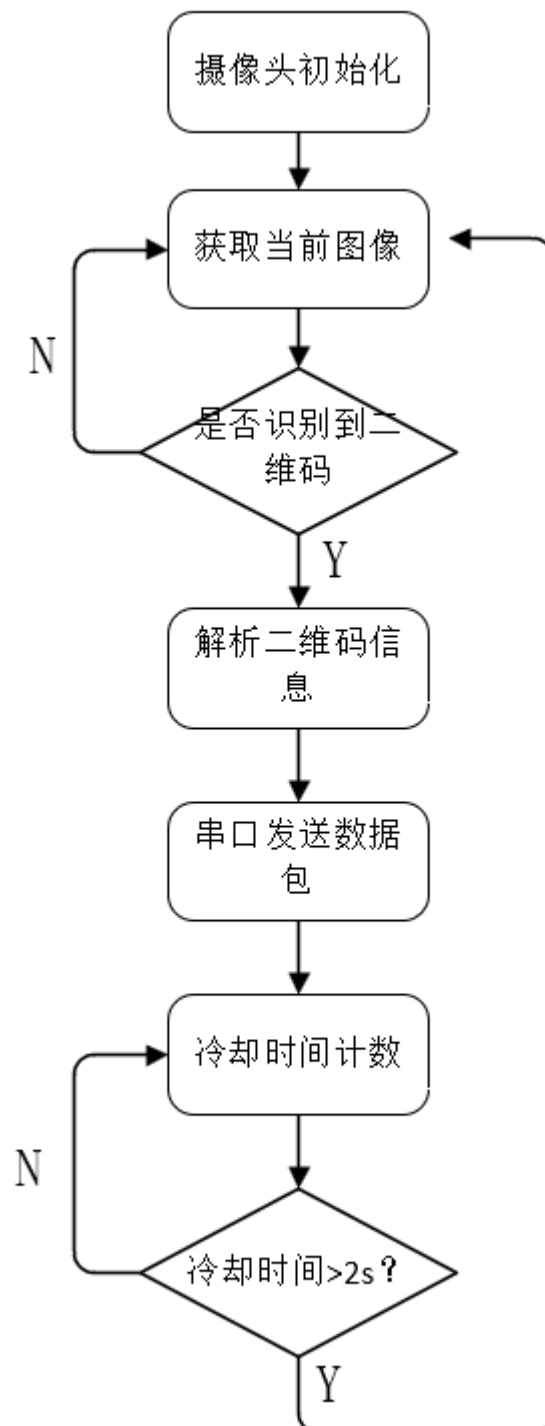
```
disp.show(img)
time.sleep_ms(1)
```

(如有其他应用层文件，按上述格式追加)

P2.2.2 功能小节划分

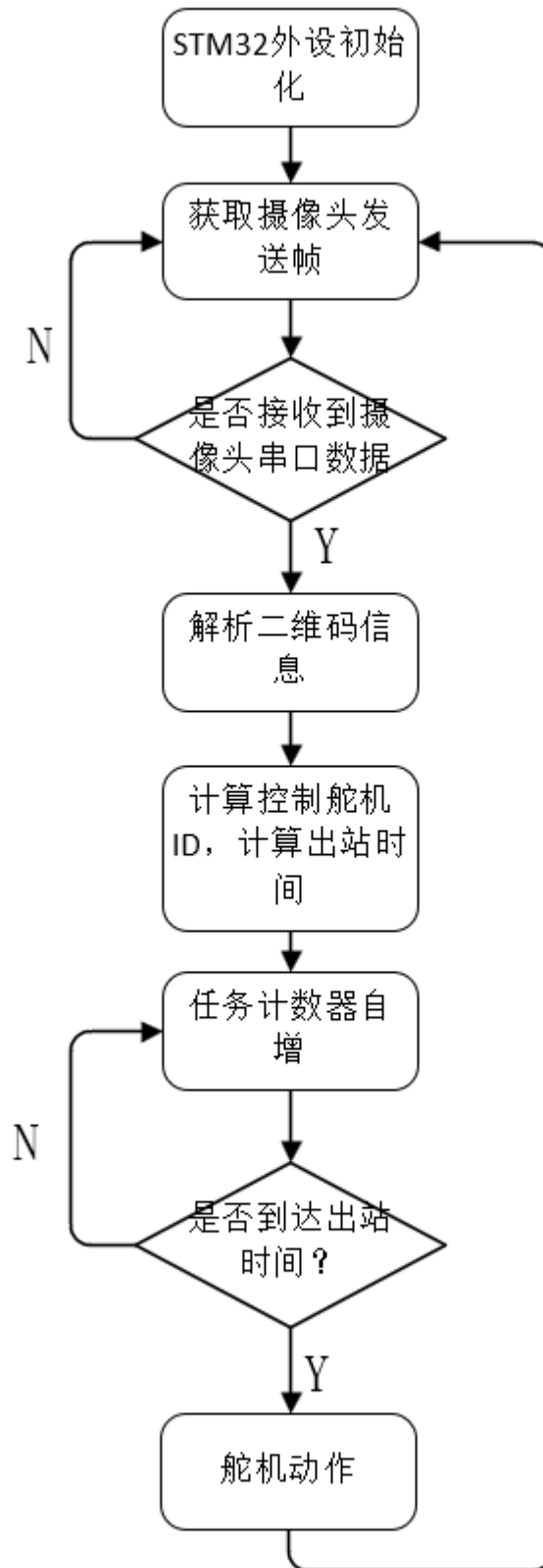
按任务书中的功能告诉 Claude 第四章 4.1.3 起的小节如何拆分。每个功能一张流程图。

功能 1：摄像头识别流程图



- 描述：摄像头识别流程图，向STM32发送数据包过程，冷却时间是为了防止同一包裹重复识别

功能 2:



- 描述：stm32端控制流程

功能 3:

- 描述:

(按需添加)

P2.2.3 上位机软件（可选）

是否包含上位机: 否

如包含:

- 描述:

通信变量分配 (remoteVar 槽位) :

TX[0]~[3] 不会到达 APP, 发送给 APP 的变量必须使用 TX[4]~[15]。

STM32 → APP (TX) :

- TX[4]:
- TX[5]:
-

APP → STM32 (RX) :

- RX[0]:
- RX[1]:
-

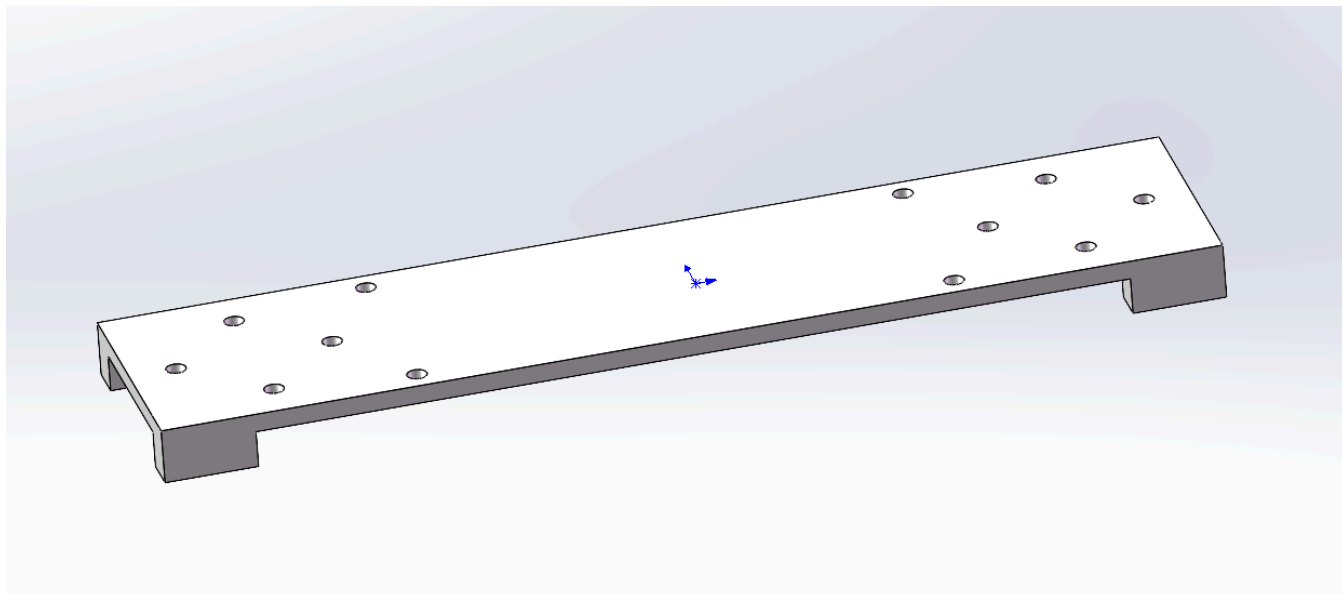
P2.3 机械结构素材（用于第五章，无则填"否"）

是否包含机械结构: 是 / 否

如包含:

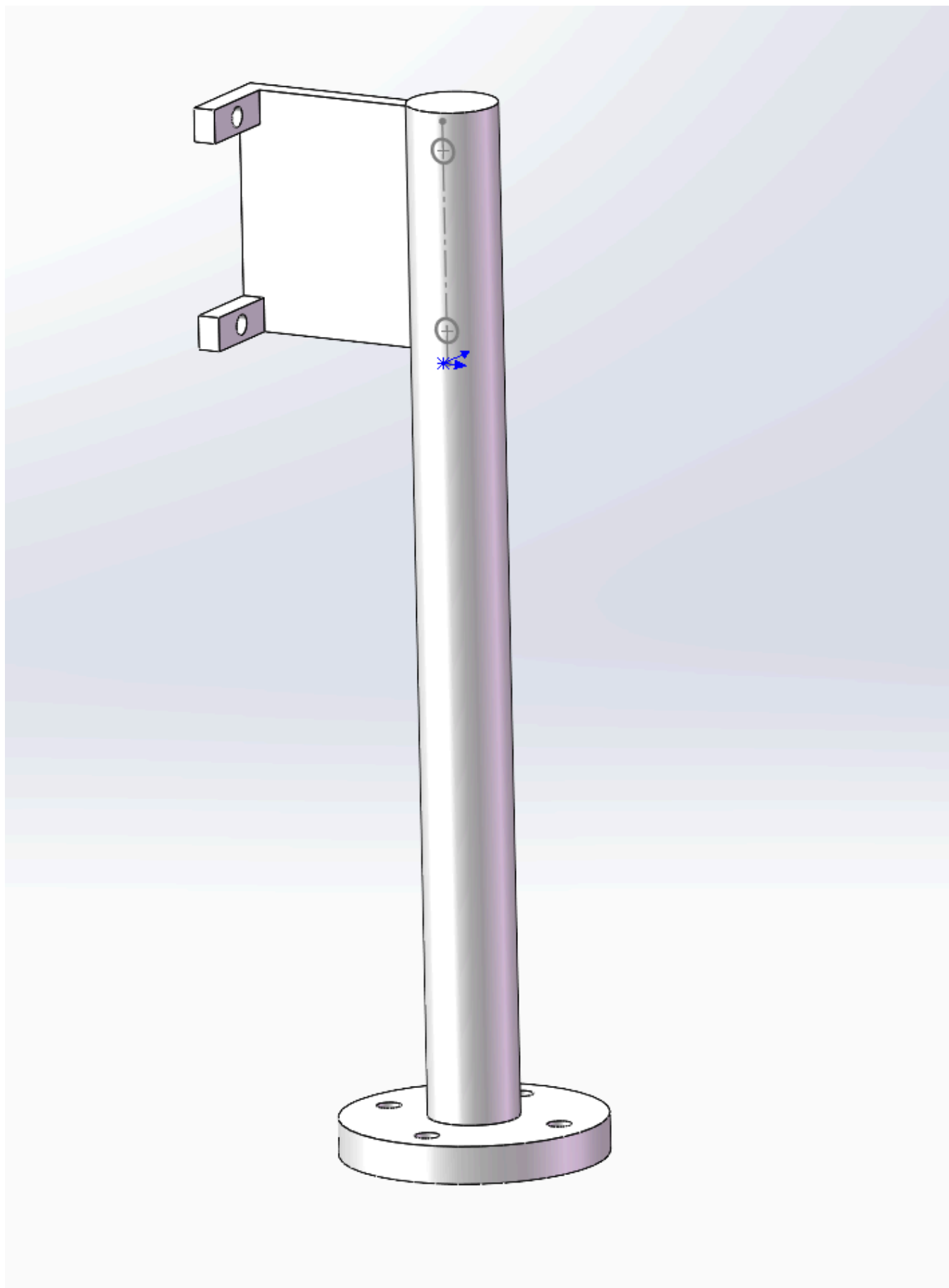
图片按 机械_1.png、机械_2.png 顺序命名。

零件 1:



- 描述：舵机固定底板

零件 2:



- 描述：舵机固定支架

零件 3:

- (按需添加更多零件)

图片处理规则：每张测试图片 Claude 必须插入论文相应小节并展开描述。

测试 1:



测试 2:



- 描述：TB6612电机驱动模块输入PWM波形，展开比30，电机调速30%

测试 3：



- 描述：板载电源5V LDO稳压

测试 4：



- 描述: 使用草料二维码生成器进行二维码生成

测试 5:



- 描述: STM32最小系统板载3v3 LDO稳压

(按需添加)

P3.2 不足与改进（可选）

诚实列 2~4 条本设计的不足。这部分老师很看重。
不填则 Claude 按项目特点自动生成 3 条通用不足。

- 1.机械结构设计的太烂，系统鲁棒性不高
- 2.
- 3.

【留空则 Claude 自由发挥】

使用说明

第 1 轮：

- 填写 G1~G5 + P1.1 + P1.2 + P1.3
- 指令： 输出 Part 1
- → 摘要 + 绪论 + 系统方案设计（含器件选型）

第 2 轮：

- 填写 P2.1~P2.3
- 指令： 输出 Part 2
- → 硬件设计（原理图分节）+ 软件设计 + 机械外观设计

第 3 轮：

- 填写 P3.1~P3.2
- 指令： 输出 Part 3
- → 系统测试 + 总结与展望

完成后指令： 整合完整论文 → Claude 输出最终单文件 md

—— 以下为项目软件实现的内部知识（Claude 自动参考，不要修改） ——

重要：以下内容仅供 Claude 撰写论文时**理解项目软件实现的内部细节**，使第四章软件设计写得专业、贴合实际代码。

⚠ 论文中绝对不允许出现以下表述：

- 不得提到"框架"、"软件框架"、"已有框架"、"项目模板"、"代码模板"等任何暗示这是基于已有框架开发的词
- 不得提到"task 层"、"driver 层"、"stdlib 层"、"func 层"、"HAL 层"等分层术语（可改写为"应用层"、"驱动层"、"基础库"、"功能模块"、"硬件抽象层"等通用工程术语）
- 不得提到 `userLib`、`userDriver`、`userFunc`、`userApp` 等具体目录名（可改写为"驱动文件目录"、"应用文件目录"等通用描述）
- 论文作者视角：所有软件结构、命名规范、调度方式都是**作者自己设计**的，不是套用的

改写示例：

- ❌ "本项目采用 task/driver/stdlib 分层框架" → ✅ "本设计采用分层式软件结构，将代码组织为应用层、驱动层和基础库三层"
- ❌ "在 task_user1 中调用 driver 层 API" → ✅ "在主应用任务中调用底层驱动接口"
- ❌ "func_appcom 模块负责通信" → ✅ "通信处理模块负责蓝牙数据收发"

1. 项目软硬件平台

- 主控芯片：STM32F103C8T6（Cortex-M3，72MHz，64KB Flash，20KB SRAM）
- 软件库：STM32 HAL 库
- 操作系统：FreeRTOS（CMSIS_V2 接口）
- 开发环境：MDK-ARM Keil 5
- 配置工具：STM32CubeMX
- 电源方案：7.4V 锂电池 → 两路 LDO 稳压输出 5V + 一路 DCDC 升压 12V（若有）

2. 软件分层结构（论文中需用通用术语改写）

项目软件按职责划分为五层（自上而下）：

内部叫法	论文中表述（建议）	职责
task 层（userApp）	应用层 / 任务层	业务逻辑，开发者主要工作区
func 层（userFunc）	功能模块层 / 中间件层	跨驱动的逻辑整合，如通信处理
driver 层（userDriver）	驱动层	各外设的驱动封装
stdlib 层（userLib）	基础库 / 工具库	通用工具函数（延时、CRC、字符串等）
HAL 层（Core/）	硬件抽象层	ST 官方 HAL 库

调用规则：上层可调用下层，禁止跨层（应用层不直接调 HAL）。

3. FreeRTOS 任务规划

项目固定使用两个 FreeRTOS 任务：

任务名	周期	职责
task_system	10ms	系统级周期任务，维护基础库状态机和通信更新
task_user1	2ms	应用级任务，承担主要业务逻辑

task_user1 内部通过 taskCnt 计数器取余实现多周期：


```
if (taskCnt % 5 == 0) { /* 10ms 周期逻辑 */ }
if (taskCnt % 50 == 0) { /* 100ms 周期逻辑 */ }
if (taskCnt % 250 == 0) { /* 500ms 周期逻辑 */ }
if (taskCnt % 500 == 0) { /* 1s 周期逻辑 */ }
```

论文中描述：可写为"主任务以 2ms 为基础调度周期，通过计数器取余实现 10ms / 100ms / 500ms / 1s 等多级时间片"。

4. 命名规范（论文中提到代码命名时遵守）

- 任务函数：xxxTaskInit() / xxxTaskUpdata()（注意是 Updata 不是 Update，这是项目代码实际拼写）
- 任务结构体：xxxTaskInfo_t + extern xxxTaskInfo_t xxxTaskInfo
- 驱动函数：DRIVER_XXX_Init() / DRIVER_XXX_Update() / DRIVER_XXX_DoSomething()
- 驱动结构体：xxxInfo 或 xxxInfo[CH_COUNT]
- 命名风格：小驼峰（变量/函数）+ 大驼峰t（结构体）+ 全大写下划线（宏）
- 注释语言：中文
- 内存分配：全静态分配，禁用 malloc

5. 通信链路（若项目有 BLE/APP 通信）

通信链路结构：

STM32 ↔ USART1 ↔ ESP32 ↔ BLE ↔ 手机 APP

- STM32 通过 USART1 与 ESP32 通信，统一使用 16 字节定长帧
- ESP32 作为 BLE 协处理器，将 STM32 数据透传给 APP，反之亦然
- 应用层通过 remoteVar_TX[16] / remoteVar_RX[16] 两个变量池读写，不需要操作底层帧

remoteVar_t 是 4 字节联合体：

```
typedef union {
    uint8_t raw[4];
    uint32_t var_uint32;
    int32_t var_int32;
    float var_float;
} remoteVar_t;
```

槽位有效宽度：

- TX/RX 索引 [0][1][4][5][8][9][12][13] → 4 字节槽位（float / uint32 / int32）
- TX/RX 索引 [2][3][6][7][10][11][14][15] → 1 字节槽位

关键约束：TX[0]~[3] 不会到达 APP（被 ESP32 截留），发送给 APP 的数据使用 TX[4]~[15]。

论文中描述：可写为"主控通过串口与无线通信模块交换数据，无线通信模块再通过低功耗蓝牙将数据透传至手机端。为简化业务层代码，作者设计了变量池机制，应用层只需读写约定的变量数组即可完成双向通信"。

6. 字节序处理（论文若涉及通信细节）

APP 端以大端序发送 4 字节字段，STM32 端 USART 接收处理函数已自动翻转为小端序。**应用层直接读写 remoteVar 即可，无需关心字节序。**

7. 上位机（若项目有 APP）

- 平台：Android 原生
- UI 框架：Jetpack Compose + Material Design 3
- 通信：BLE GATT
- 主页面：单页 HomeScreen，包含项目名称/作者标题区 + 多个分区
- UI 组件：分区用 HomeSection，数据行用 ProtoFieldRow（标签：值），控制用 Button
- 设计原则：
 - STM32 → APP 的数据 → 文本显示
 - APP → STM32 的控制 → 按钮（不用 Slider/SeekBar）
 - 数值类控制用多个按钮代替（如 [音量+] [音量-] 或 [亮度20%] [亮度50%]）

论文中描述：可写为"上位机界面采用现代化的声明式 UI 框架开发，将界面划分为若干功能分区，数据显示采用文本形式，控制操作统一使用按钮实现，避免滑动控件可能带来的误触和精度问题"。

8. 论文写作指引

撰写第四章软件设计时，Claude 应当：

1. **从作者视角讲述设计思路**，不要写成"项目使用了 XX 框架"——而是"作者经过分析后采用了 XX 设计方案"
2. **代码命名直接使用项目实际命名**（user1TaskInfo、DRIVER_XXX_Init 等），但解释命名时说"作者按照模块化原则采用统一的命名风格"
3. **分层结构用通用工程术语描述**，不要出现 task/driver/stdlib 这种内部叫法
4. **2ms 基础周期 + taskCnt 取余**这套机制要解释为"作者的设计巧思"，而不是"框架要求"
5. **遇到代码细节时**，结合代码本身解释作者的实现思路，让论文显得是真的写过这份代码